📄 Comprehensive Project Report: AI Wealth Visualiser

1. Executive Summary

The AI Wealth Visualiser is a lightweight, high-performance command-line application designed to democratize advanced financial planning. Moving away from bloated web applications and complex Excel spreadsheets, this tool delivers institutional-grade financial forecasting—including Monte Carlo market simulations—directly to the user's terminal. It is built on a strict privacy-first architecture, ensuring that sensitive personal financial data never leaves the local machine.

2. Core Philosophy & Objectives

- Absolute Privacy: In an era of rampant data harvesting, this application operates entirely offline. No APIs, no telemetry, and no cloud storage.
- Algorithmic Transparency: Users are shown exactly how their wealth compounds, how inflation erodes purchasing power, and how market volatility impacts their retirement timelines.
- Terminal-Native UI/UX: Proving that terminal applications can be beautiful, the project leverages dynamic ANSI escape codes to render responsive charts, graphs, and sparklines without requiring a graphical user interface (GUI).

---

3. Feature Deep-Dive & System Mechanics

A. Dynamic Cashflow Assessment

The application begins by establishing a financial baseline. It interactively collects data across multiple income streams and granular expense categories.

- The Math: It immediately calculates the user's Savings Rate and cross-references it against established financial independence benchmarks (e.g., flagging anything below 20% as a risk, and 50%+ as "FIRE Zone").

B. Intelligent Wealth Projection Engine

Unlike basic compound interest calculators, this module splits the user's net worth into distinct vectors:

- Current Corpus Growth: How existing assets (savings + investments) will grow over the target years.
- Future SIP Contributions: How ongoing monthly investments will compound.

- Inflation Adjustment: Automatically applies a standard 6.2% inflation rate to project what the user's *current* living expenses will cost in the future, providing a realistic target rather than an illusionary number.

C. The Monte Carlo Simulation Engine (The Crown Jewel)

Financial markets are not linear. To stress-test the user's retirement plan, the application utilizes the numpy library to run 1,000 parallel market simulations.

- Mechanism: It applies random Gaussian distributions based on the chosen investment profile's Expected Return (μ) and Volatility (σ).
- Output: It generates a probability of success (e.g., "75% chance of hitting your target") and displays the outcomes across the 10th (worst-case), 50th (median), and 90th (best-case) percentiles using dynamic terminal sparklines.

D. Heuristic "AI" Insights System

The script features a rule-based logic engine that analyzes the collected data points to generate custom, color-coded financial advice.

- Triggers: It actively scans for critical red flags, such as an insufficient emergency fund (less than 6 months of expenses), high debt-to-income ratios (loan repayments exceeding 30% of income), or a mathematical shortfall in required SIP contributions.

---

4. Technical Architecture & Specifications

- Language: Python 3.x
- External Dependencies: numpy (Strictly used for high-performance array calculations and random number generation for the Monte Carlo simulations).
- Interface Rendering: Custom implementation of ANSI color codes and ASCII characters (_[redacted]) to generate:
    - Vertical bar charts for income/expense breakdowns.
    - Multi-series time-lapse line charts mapped to terminal grids.
    - Distribution histograms for final wealth probability.
- Responsive Design: Utilizes shutil.get_terminal_size() to dynamically scale charts and center text based on the user's current terminal window dimensions (capped at 110 columns for readability).

- Concurrency: Implements Python threading to run the heavy 1,000-run simulation in the background while rendering a smooth loading spinner in the foreground UI.

---

5. Target Audience & Use Cases

- Software Engineers & Developers: Users who live in the terminal and prefer highly technical, fast, and local tools over web apps.
- FIRE Movement Enthusiasts: Individuals aggressively pursuing Financial Independence, Retire Early (FIRE) who need to run complex timeline and safe-withdrawal-rate scenarios.
- Privacy Advocates: Anyone who wants detailed financial planning without linking their bank accounts or entering salaries into a third-party website.

---

6. Future Roadmap (Proposed Enhancements)

While v2.0 is highly capable, future iterations could include:

1. Tax Drag Calculations: Factoring in Long-Term Capital Gains (LTCG) tax dynamically based on the withdrawal year.
2. Export Functionality: Allowing the user to dump the final analysis into a local .pdf or .md report.
3. Custom Asset Allocation: Permitting users to blend profiles (e.g., 60% Equity / 40% Debt) rather than picking a single track.